



**МИНОБРНАУКИ РОССИИ**

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

**«МИРЭА – Российский технологический университет»**

**РТУ МИРЭА**

---

---

**Институт информационных технологий (ИИТ)**

**Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5**

по дисциплине «Системы управления базами данных»

Студент группы *ИНБО-20-23. Школьный И.В.*

---

(подпись)

Преподаватель *Трушин С.М.*

---

(подпись)

Москва 2025 г.

## СОДЕРЖАНИЕ

|                                          |                                        |
|------------------------------------------|----------------------------------------|
| ПОСТАНОВКА ЗАДАЧИ.....                   | 3                                      |
| 1 ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ .....        | 4                                      |
| 1.1 Подготовка среды.....                | <b>Ошибка! Закладка не определена.</b> |
| 1.2 Создание первого потока данных ..... | <b>Ошибка! Закладка не определена.</b> |
| 1.3 Создание простого конвейера .....    | <b>Ошибка! Закладка не определена.</b> |
| 1.4 Проверка результатов .....           | <b>Ошибка! Закладка не определена.</b> |
| 1.5 Создание шаблона потока .....        | <b>Ошибка! Закладка не определена.</b> |
| 1.6 Работа с группами обработки .....    | <b>Ошибка! Закладка не определена.</b> |
| 1.7 Самостоятельная работа .....         | <b>Ошибка! Закладка не определена.</b> |
| ВЫВОД.....                               | 13                                     |

# ПОСТАНОВКА ЗАДАЧИ

Тема: Работа с Apache Kafka: создание тем, производителей, потребителей и интеграция с Flume.

Цель работы: освоить базовые операции работы с Apache Kafka: создание тем, запуск производителей и потребителей, отправка сообщений в потоковом и пакетном режимах. Изучить интеграцию Flume и Kafka для построения конвейера потоковой обработки данных.

Оборудование и ПО:

- операционная система Linux (Ubuntu или аналогичная);
- Apache Kafka;
- Apache Zookeeper;
- Apache Flume;
- Python 3;
- терминал Linux;
- текстовый редактор (nano, vim, gedit и др.).

Файлы, применяемые в ходе выполнения работы:

- `alice_in_wonderland.txt` — текстовый файл, содержащий данные для отправки в Kafka (как вручную, так и в пакетном режиме);
- `spool_stream.py` — скрипт Python для генерации потоковых файлов и их передачи в каталог буферизации (spool);
- `server_logs.txt` — набор строк серверных логов, используемых для моделирования реального потока логов;
- `spool_stream_extended.py` — модифицированный скрипт генерации потоков для расширенной задачи.

# 1 ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Apache Kafka — распределённая платформа потоковой передачи данных, используемая для публикации, хранения, обработки и подписки на потоки событий в реальном времени. Основные компоненты:

1. Тема (topic) — логическая сущность, в которую записываются сообщения.
2. Производитель (producer) — приложение или сервис, отправляющий сообщения в Kafka.
3. Потребитель (consumer) — приложение, считывающее сообщения из темы.
4. Zookeeper — служба координации, используемая Kafka для управления кластером.
5. Flume — инструмент для сбора, агрегации и передачи больших объемов данных, поддерживающий интеграцию с Kafka через Kafka Sink.

Работа с Kafka часто включает:

- создание тем;
- запуск консольных производителей и потребителей;
- передачу сообщений в реальном времени и в пакетном режиме;
- построение конвейеров данных с использованием Flume.

## 1.1 Знакомство с сервисом

Приостановим сервис HBase. Перезапустим kafka и zookeeper и убедимся, что оба сервиса запущены (Рисунок 1).

```
[student@192 ~]$ sudo stop-hbase.sh
no hbase master found
[student@192 ~]$ sudo systemctl stop kafka
[student@192 ~]$ sudo systemctl stop zookeeper
[student@192 ~]$ sudo systemctl start zookeeper
[student@192 ~]$ sudo systemctl start kafka
[student@192 ~]$ sudo systemctl status kafka
● kafka.service
   Loaded: loaded (/etc/systemd/system/kafka.service; disabled; vendor preset: disable
d)
   Active: active (running) since Tue 2025-12-02 19:22:17 KST; 8s ago
     Main PID: 8829 (sh)
        CGroup: /system.slice/kafka.service
                └─8829 /bin/sh -c /home/kafka/kafka/bin/kafka-server-start.sh /home/kafk...
                  └─8831 java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -...

Dec 02 19:22:17 192.168.1.6 systemd[1]: Started kafka.service.
[student@192 ~]$ sudo systemctl status zookeeper
● zookeeper.service
   Loaded: loaded (/etc/systemd/system/zookeeper.service; disabled; vendor preset: dis
abled)
   Active: active (running) since Tue 2025-12-02 19:22:11 KST; 23s ago
     Main PID: 8439 (java)
        CGroup: /system.slice/zookeeper.service
                └─8439 java -Xmx512M -Xms512M -server -XX:+UseG1GC -XX:MaxGCPauseMillis=...

Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,0...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,1...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,1...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,1...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,1...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,2...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,2...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,2...
Dec 02 19:22:13 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:13,2...
Dec 02 19:22:20 192.168.1.6 zookeeper-server-start.sh[8439]: [2025-12-02 19:22:20,2...
Hint: Some lines were ellipsized, use -l to show in full.
[student@192 ~]$
```

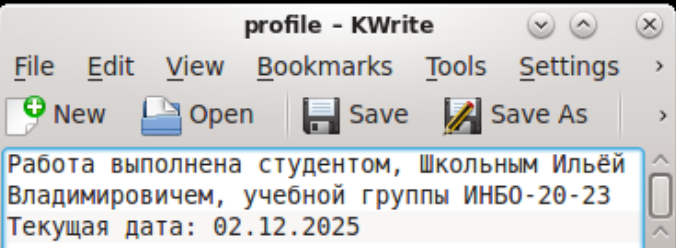


Рисунок 1 — Перезапуск сервисов kafka и zookeeper

Создадим тему topic1\_logs. Выведем список всех созданных тем и посмотрим параметры созданной темы (Рисунок 2).

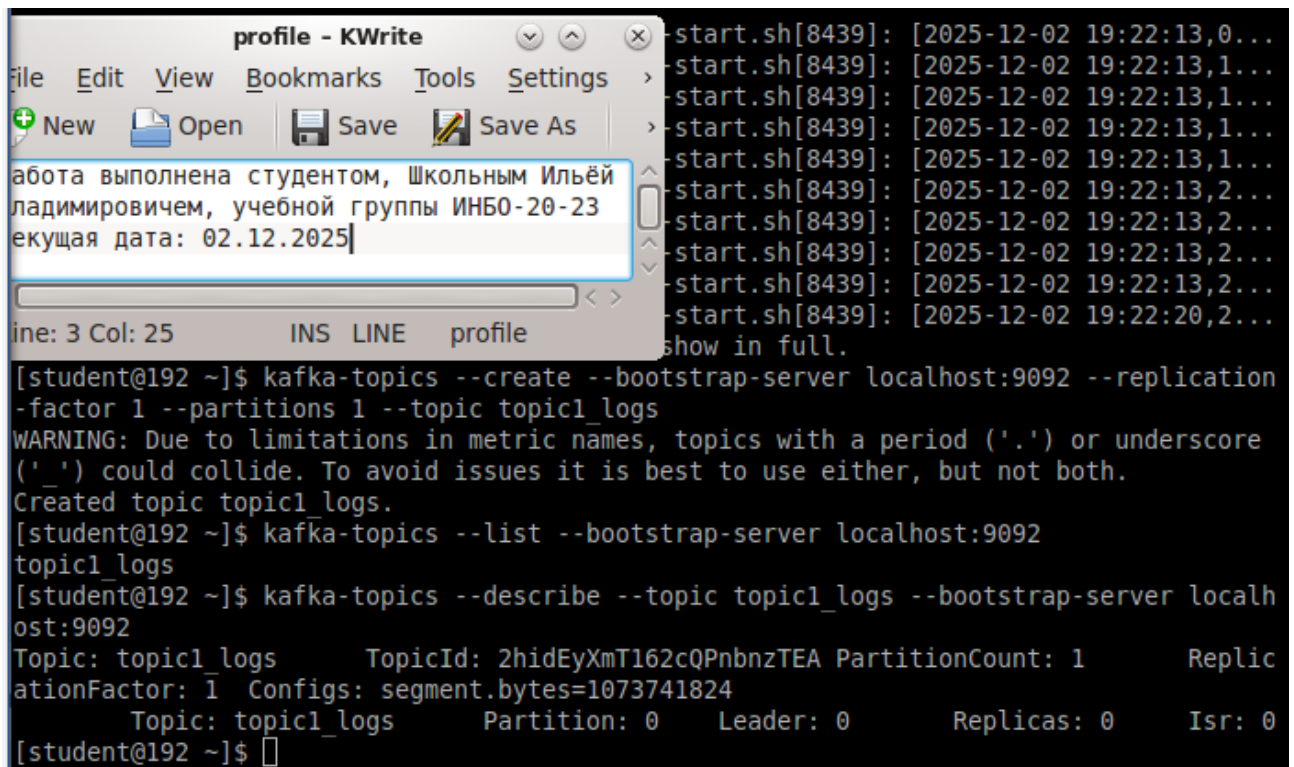


Рисунок 2 — Знакомство с темами

Откроем новый терминал и запустим в нем producer. Во втором новом терминале запустим consumer. Убедимся в корректности: при вводе в терминал producer, это же самое сообщение должно появиться в терминале consumer (Рисунок 3).

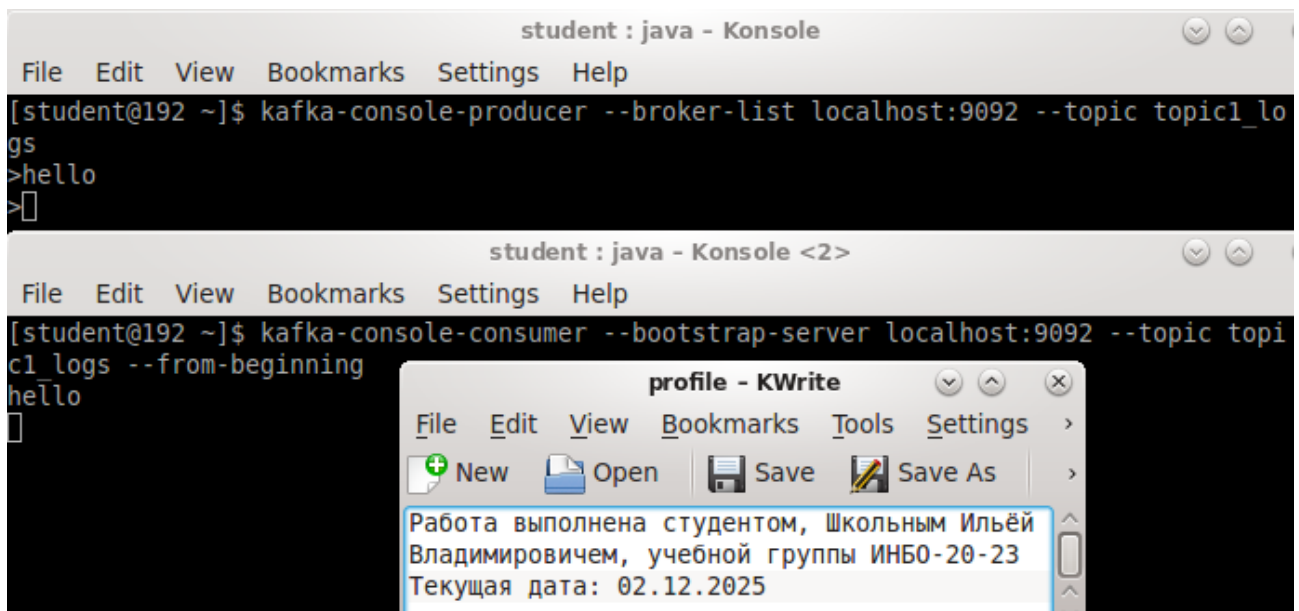


Рисунок 3 — Producer и Consumer

Остановим работу производителя. Передадим в поток содержимое файла `alice_in_wonderland.txt` и заметим, что потребитель мгновенно обработал этот поток (Рисунок 4).

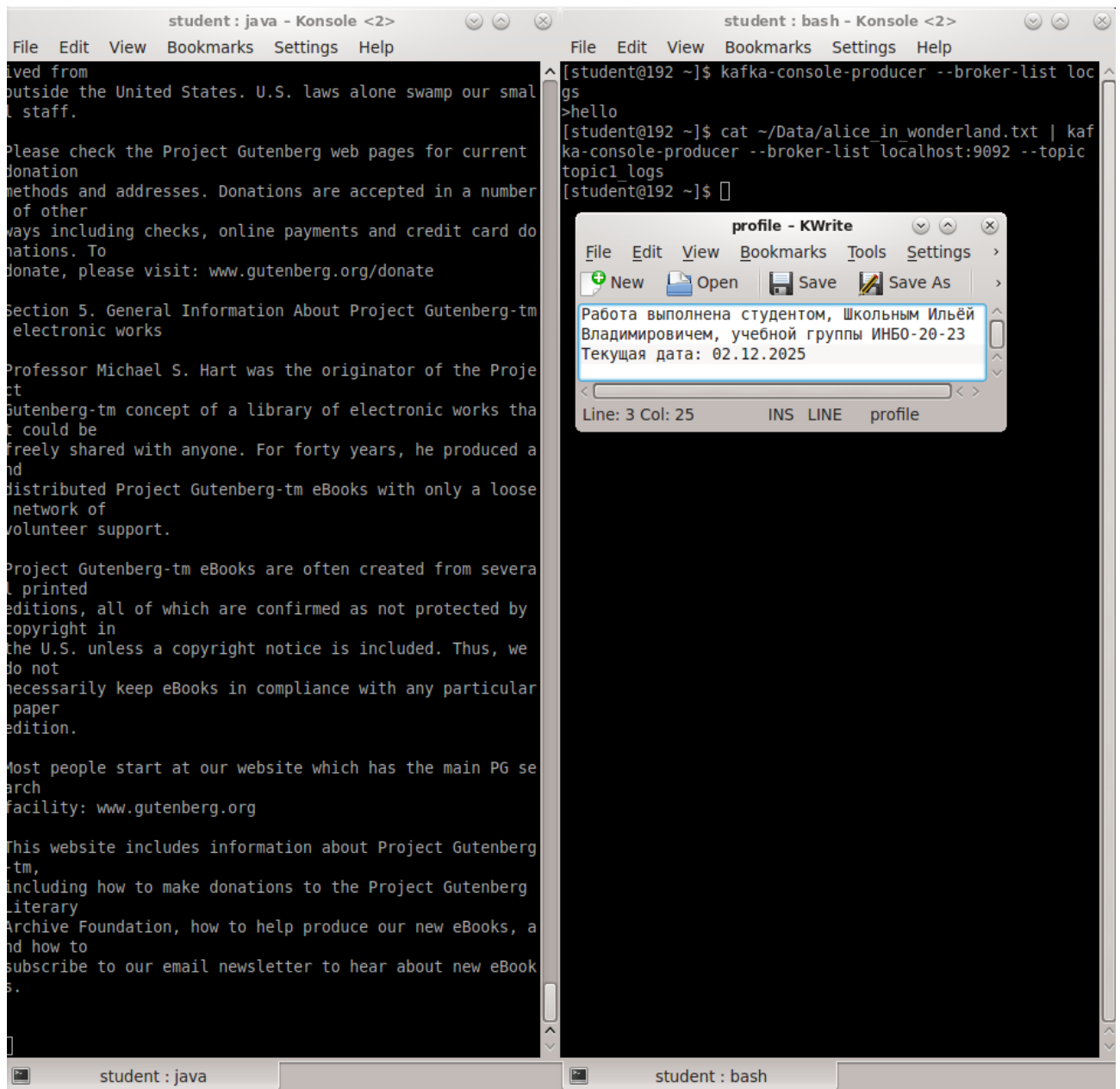
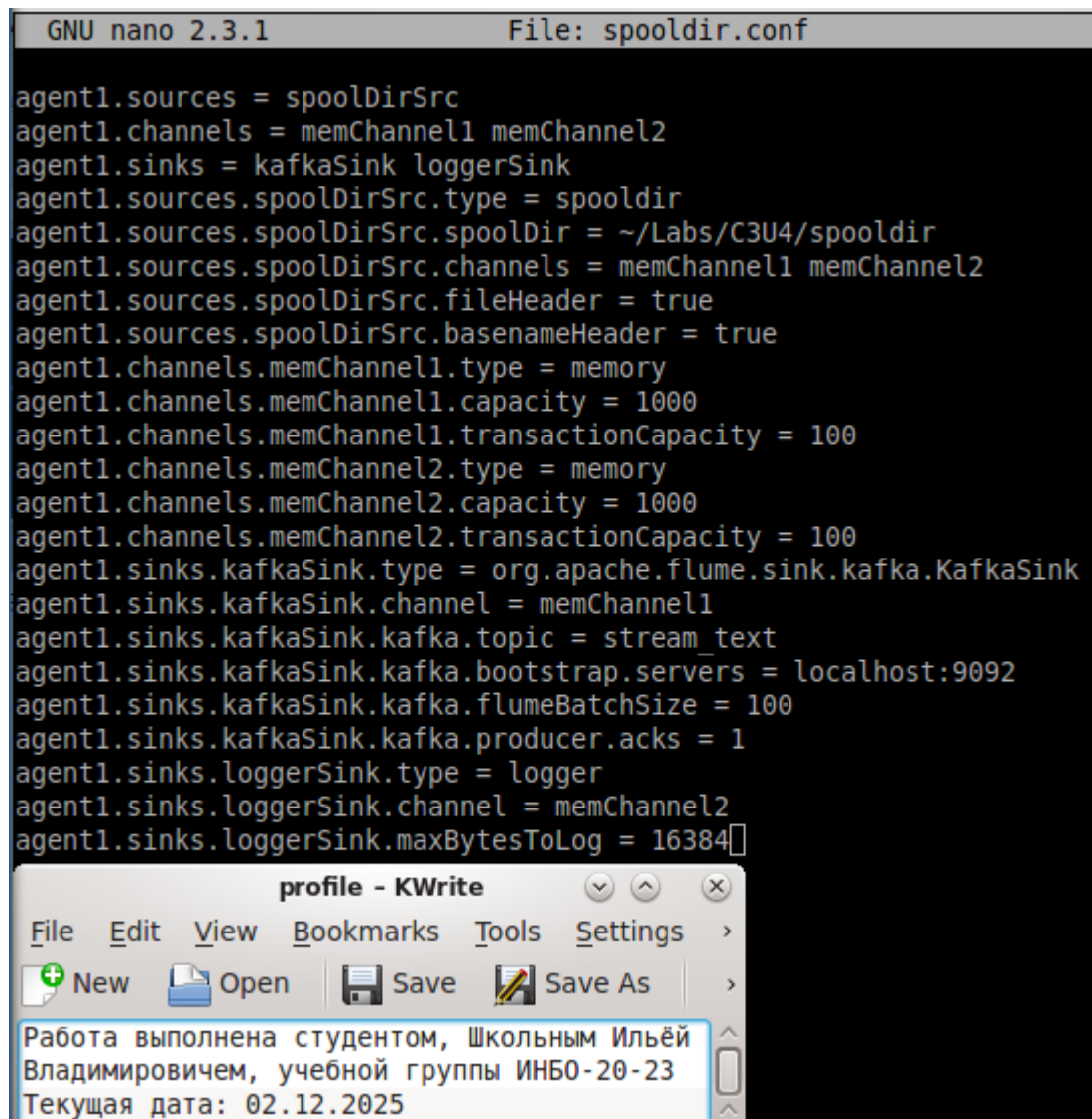


Рисунок 4 — Передача содержимого файла в поток

Перейдем в каталог `~/Labs/C3U4` и создадим в ней каталог `spooldir`. Затем напишем конфигурационный файл согласно требованиям (Рисунок 5).



**Рисунок 5 — Создание spooldir.conf**

Создаем тему stream\_text по предыдущему образцу. Запускаем потребителя. Затем удаляем и создаем каталог spool. Запускаем flume агента и симулятор потоков. Видим, как создаются файлы (Рисунок 6).

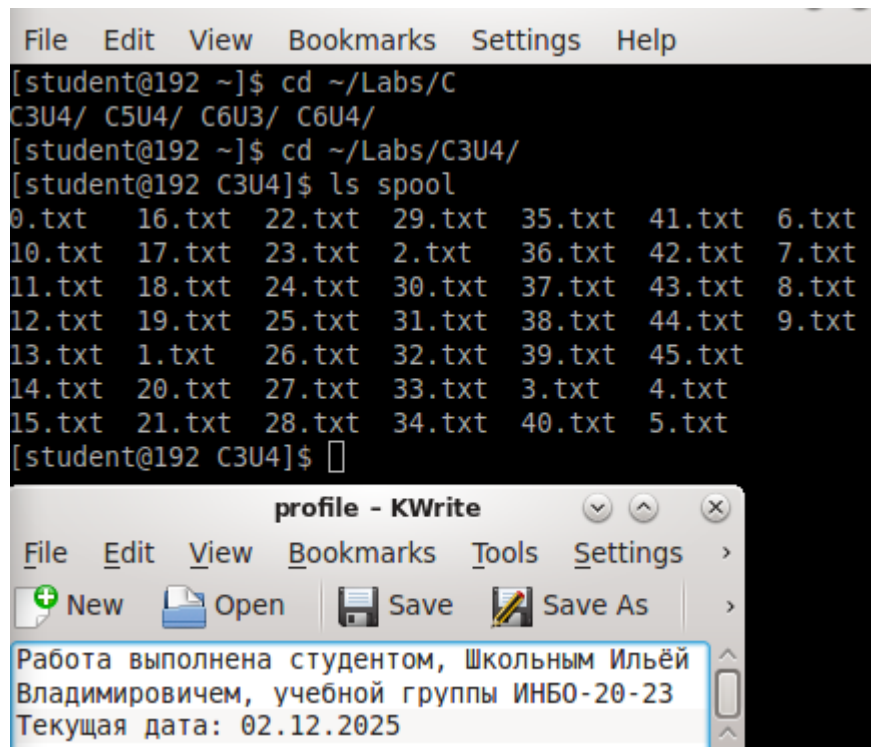


Рисунок 6 — Поток файлов

Ответим на контрольные вопросы:

1. Тема — логический канал/категория, в которую производители отправляют сообщения, а потребители из нее читают.
2. Zookeeper координирует работу кластера Kafka: хранит метаданные, отслеживает состояние брокеров, управляет контроллером и конфигурацией.
3. Производитель (producer) пишет данные в топик, потребитель (consumer) читает данные из топика.
4. Отправка группы сообщений за один раз (например, из файла), а не по одному в реальном времени.
5. Отслеживает указанную директорию, читает новые файлы целиком, переименовывает их после обработки (добавляет .COMPLETED).
6. Flume собирает и агрегирует данные из различных источников, Kafka обеспечивает надежную буферизацию, масштабируемость и потоковую обработку.
7. Компонент Flume, который записывает события из канала Flume в топик Kafka.

8. Capacity — максимальное количество событий в канале, transactionCapacity — количество событий за одну транзакцию. Контролируют производительность и надежность.
9. Через Kafka Sink: Flume читает данные из источника, потом помещает в канал, а Kafka Sink забирает из канала и отправляет в топик Kafka.
10. Чтобы увидеть все сообщения в топике с начала, а не только новые, которые приходят после запуска consumer.

## 1.2 Самостоятельная работа

Создадим топик `server_log_stream` все по тому же образцу. Далее создаем конфигурационный файл (Рисунок 7).

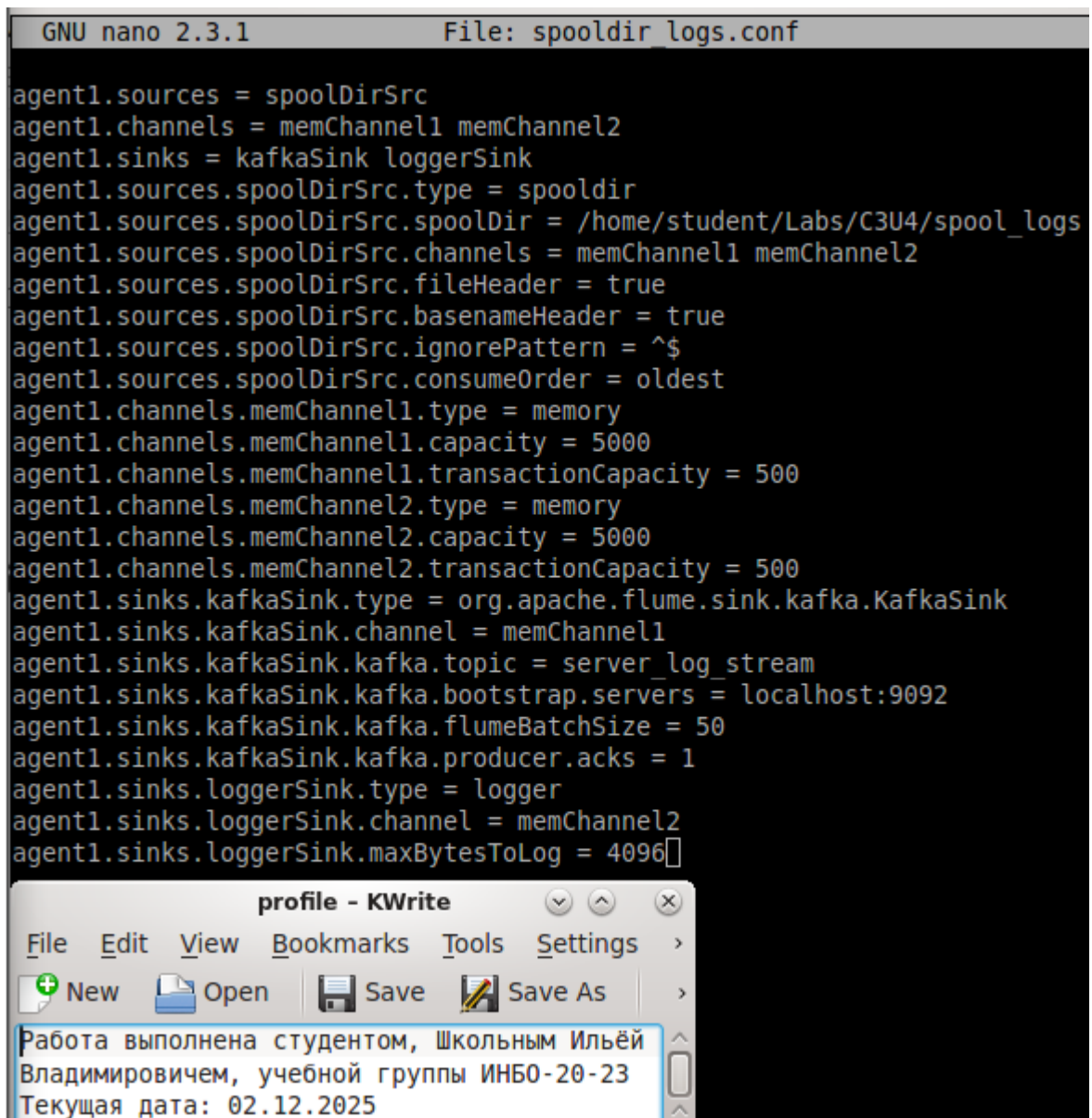


Рисунок 7 — spooldir\_logs.conf

Напишем скрипт `spool_stream_extended.py`, чтобы тот читал файл `server_logs.txt`, каждые 3 секунды создавал небольшой batch логов, помещал их в каталог `spool`. Запускаем агента, потребителя и скрипт и наблюдаем результат (Рисунок 8).

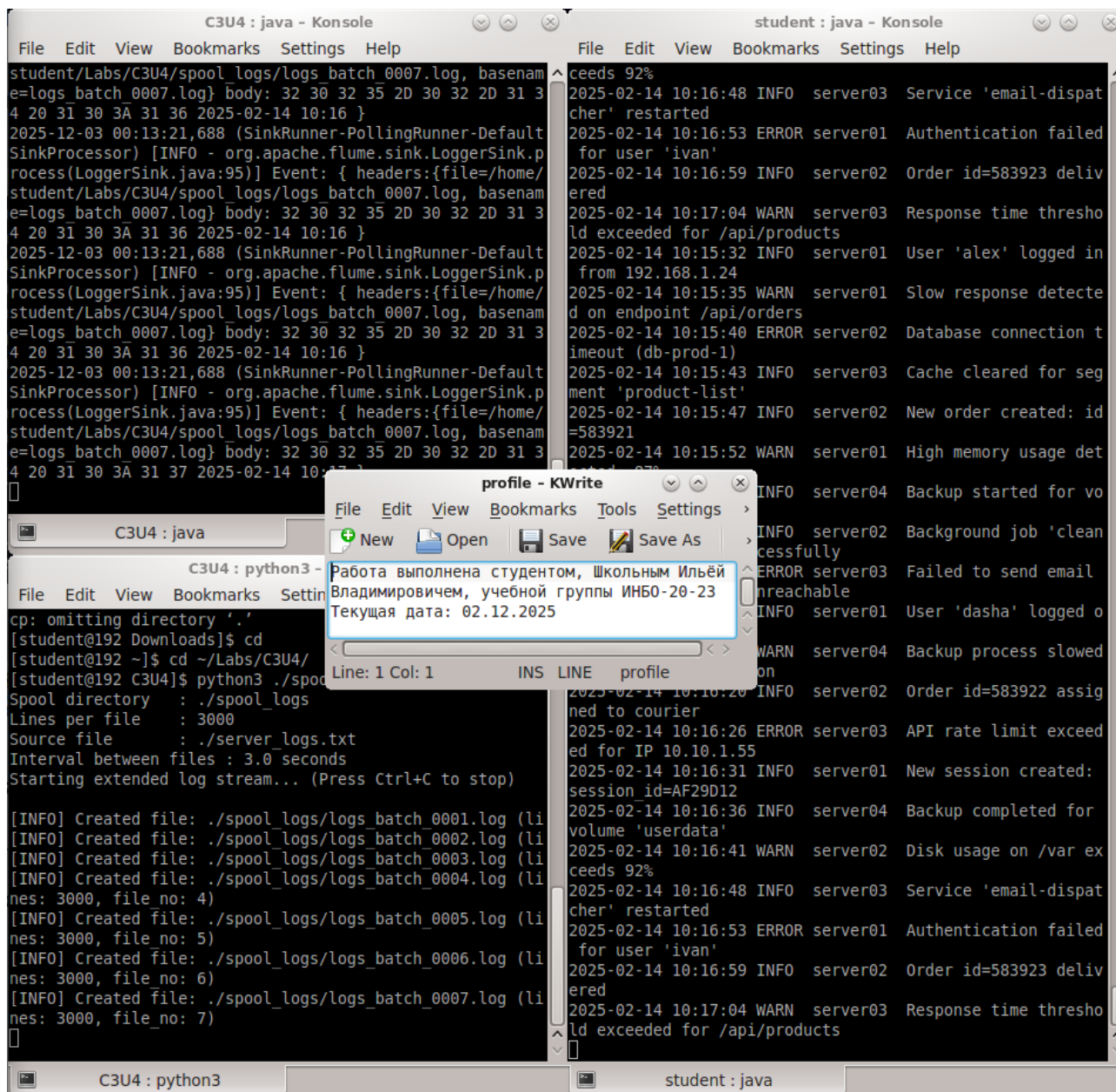


Рисунок 8 — Корректная работа программы

## **ВЫВОД**

В ходе выполненной работы была изучена практическая настройка и отладка потоков данных в Apache Kafka.